Memorando

Aplicações Móveis, 2026/2027

De: Emanuel Carneiro dos Santos
Nº de Matrícula: 20230429
Assunto: Introdução a Linguagem Kotlin
Data: 20/10/2026

1. Introdução

O presente laboratório tem como objectivo introduzir a linguagem Kotlin, recomendada pela Google para o desenvolvimento de aplicações Android.

O **Kotlin** é uma linguagem moderna que corre sobre a JVM e é totalmente interoperável com Java. Destaca-se pela sintaxe concisa, pela *null safety* (tipos que admitem ou não o valor null, verificados em tempo de compilação), por recursos funcionais como lambdas e operações sobre colecções, por funções de extensão e ainda por classes de dados (*data classes*) e tipos selados (*sealed classes*). Sendo um programador com experiência prévia em Java, a transição foi facilitada pela semelhança do modelo de objectos e do ecossistema de ferramentas.

No âmbito desta experiência laboratorial, foram abordados os seguintes tópicos: variáveis (val, var), tipos básicos, *string templates* ($) e *null safety*; colecções (List, Set e Map); funções e lambdas; controlo de fluxo (if, when, for, while e do-while); e orientação a objectos (classes, data class, classes abstractas, interfaces, herança, polimorfismo) e organização em pacotes.

O trabalho foi dividido em duas fases. Na primeira, o sistema foi implementado de forma procedimental, com uma função por comando (i, l, r, b, m, v, w). Na segunda, o código foi refactorado segundo o paradigma orientado a objectos e estendido para suportar vários utilizadores, identificados por email e password, cada um com os seus próprios movimentos e sessões. Como restrições, assumiu-se que todas as datas são inteiros entre 0 e 999, que existem no máximo 100 movimentos por data e que a descrição de cada movimento não excede 60 caracteres.

2. Experiências Realizadas

Esta secção descreve a configuração do ambiente e a implementação incremental da aplicação de consola «Sistema de Orçamento Pessoal» (apêndice A1 do guia), seguindo a sequência proposta no enunciado.

2.1. Configuração do ambiente

Foi instalado o JDK 21 (Oracle OpenJDK 21.0.12) e o IntelliJ IDEA. O projecto foi criado através do assistente **New Project** (Figura 1) com a linguagem **Kotlin**, o nome **Hello**, o sistema de construção **Gradle** com DSL Kotlin, o JDK 21 e a opção *Add sample code*, que gera o ficheiro `Main.kt` com a função `main`. Não se marcou *Create Git repository*, porque o projecto fica dentro da pasta do repositório da disciplina (`Laboratorio01/Hello`). A distribuição do Gradle é o *Wrapper* (versão 9.6.0, seleccionada automaticamente), pelo que a construção não depende de uma instalação local; foi necessário acesso à internet para descarregar o Gradle e as dependências.

O ficheiro `build.gradle.kts` declara o plugin `kotlin("jvm")` na versão 2.4.20, o repositório `mavenCentral()`, a dependência de testes `kotlin("test")` com JUnit Platform e `jvmToolchain(21)`, que fixa a versão do JDK usada na compilação. O pacote base do código é `org.emanueldossantos`.

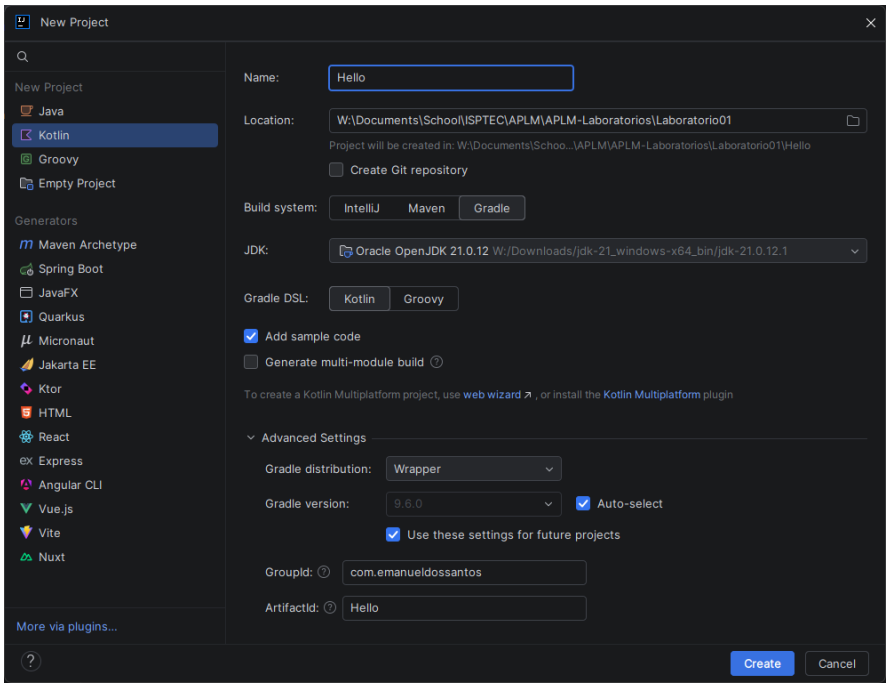


Figura 1: Assistente New Project do IntelliJ IDEA com a configuração do projecto Hello.

2.2. Ciclo de leitura de comandos

Declarou-se a variável `cmd` do tipo `String` (`var`, pois é reatribuída em cada iteração) e criou-se um ciclo `do-while` que lê uma linha da consola com `readln()`, a guarda em `cmd` e a apresenta com `println(cmd)`, terminando quando o utilizador escreve `exit` (Figura 2). Como `String` é um tipo não anulável e `readln()` nunca devolve `null` (ao contrário de `readlnOrNull()`), não é necessário qualquer tratamento de *null safety* neste ponto. Na Figura 2 cada linha introduzida (verde, itálico) é seguida do respectivo eco.

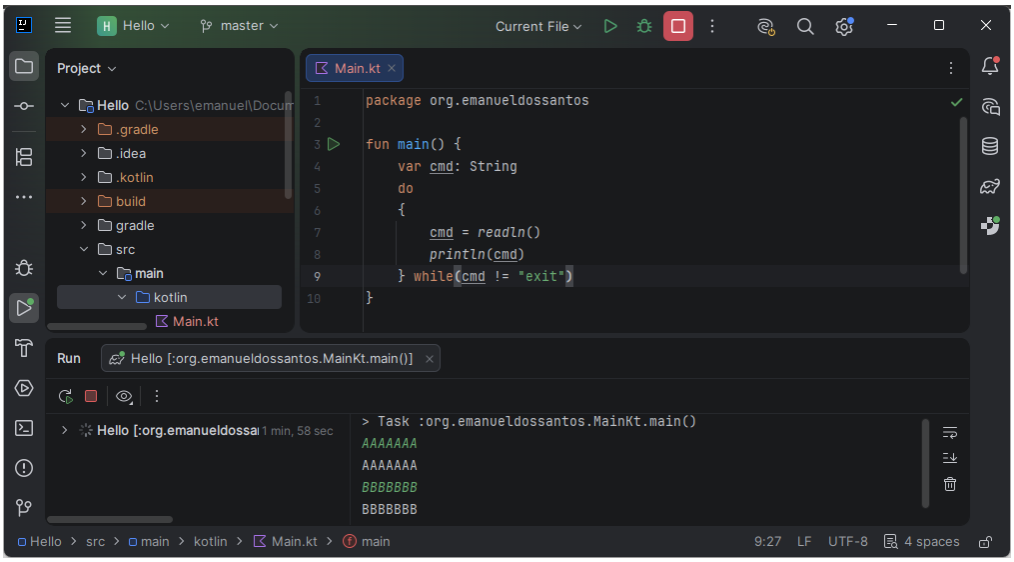


Figura 2: Ciclo do-while de leitura (readln) e eco (println) até ao comando exit.

2.3. Identificação e despacho dos comandos

Em seguida implementou-se `getCommand(cmd: String): String`, que separa a linha por espaços com `split(" ")` e devolve o primeiro elemento, isto é, a letra do comando. A Figura 3 mostra que, para a linha `i 200 0 30 r Mesada`, o programa devolve `i` e, para `b 20`, devolve `b`.

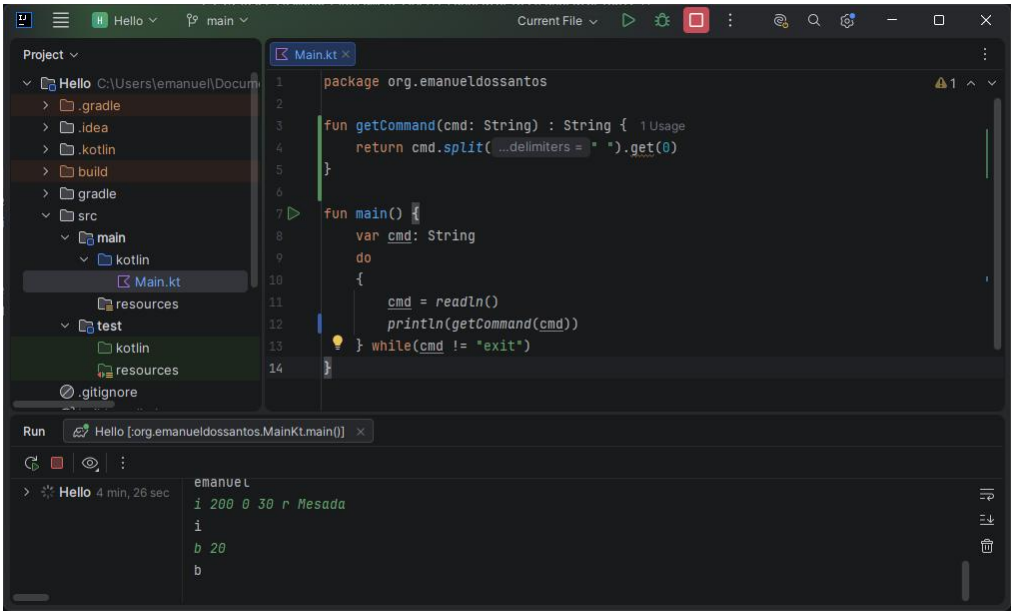


Figura 3: Função getCommand e identificação do comando introduzido.

Esta versão evoluiu para `execCommand(cmd: String)` (Figura 4): ignora linhas vazias (`isBlank()`), normaliza a entrada com `trim()` e `split(Regex("\\s+"))`, tolerando espaços múltiplos, e despacha para a função de cada comando através de `when (args[0])`, que em Kotlin dispensa o `break` do `switch` do Java. O ramo `else` escreve «cmd: comando desconhecido!». Os argumentos são convertidos com `toInt()` e passados como parâmetros nomeados (`montante =`, `data =`, …), o que torna a chamada auto-explicativa. O ciclo principal passou a `while (true)` com `break` quando `cmd == "exit"`.

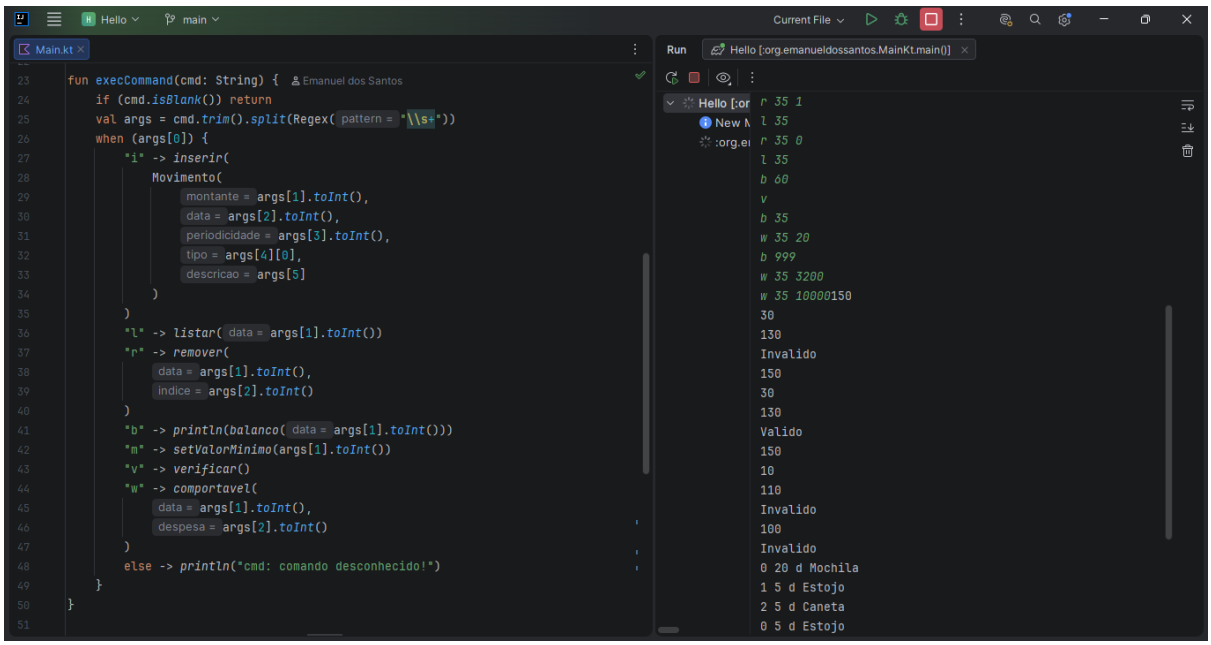


Figura 4: Função execCommand (despacho com when) e execução com os dados de entrada do exemplo do enunciado.

2.4. Estruturas de dados

O movimento foi modelado como uma `data class`, que gera automaticamente `equals()`, `hashCode()`, `toString()` e `copy()`; como todas as propriedades são `val`, os objectos são imutáveis. O estado do programa fica em duas variáveis ao nível do ficheiro: `lista`, uma `MutableList<Movimento>`, e `valueMinimo`, inicializado a 0 (valor por defeito do enunciado).

```
data class Movimento(
    val montante: Int,
    val data: Int,
    val periodicidade: Int,
    val tipo: Char,          // 'r' receita, 'd' despesa
    val descricao: String
)

val lista = mutableListOf<Movimento>()
var valueMinimo: Int = 0
```

Optou-se por uma lista porque (a) preserva a ordem de inserção, exigida na listagem; (b) admite elementos repetidos; e (c) o índice de um movimento é a sua posição relativa entre os movimentos da mesma data, obtida com `filter { it.data == data }`, pelo que, ao remover um movimento, os índices dos restantes ficam automaticamente actualizados. Os movimentos periódicos são expandidos na inserção: `inserir` cria um `Movimento` por cada ocorrência (`data`, `data + p`, `data + 2p`, … até 999) com `periodicidade = 0`, já que o estado interno não distingue movimentos periódicos de únicos, tal como pede o enunciado.

2.5. Implementação dos comandos

Cada comando foi implementado numa função própria, pela ordem sugerida no guia (Tabela 1).

Cmd	Função	Descrição
`i`	`inserir(movimento)`	Insere o movimento; se `periodicidade > 0` repete-o de p em p datas até 999 (`do-while`).
`l`	`listar(data)`	Filtra por data e escreve `índice montante tipo descrição` com `forEachIndexed`.
`r`	`remover(data, indice)`	Remove o movimento de índice dado entre os da data, se `indice in movimentos.indices`; retira apenas um movimento.
`b`	`balanco(data)`	Soma, com `sumOf`, as receitas e subtrai as despesas de todos os movimentos com `data` ≤ à indicada.
`m`	`setValorMinimo(valor)`	Actualiza o valor mínimo de balanço (por defeito 0).
`v`	`verificar()`	Escreve `Valido` se o balanço de todas as datas (0 a 999) for ≥ ao mínimo; caso contrário `Invalido`.
`w`	`comportavel(data, despesa)`	Escreve a primeira data a partir de `data` em que a despesa não invalida o orçamento, ou `Incomportavel`.

Tabela 1: Comandos e funções que os implementam.

Os comandos `v` e `w` partilham a função `calcularBalancos()`, que acumula os montantes por data num `IntArray(1000)` (somas acumuladas) e devolve o balanço de cada data. Uma despesa na data *d* reduz o balanço em todas as datas ≥ *d*, pelo que `comportavel` testa cada data candidata *d* ≥ `data` e devolve a primeira em que `balancos[f] - despesa >= valueMinimo` para todo o *f* em *d*..999. No pior caso são cerca de 10^6 comparações, aceitável para os limites do problema.

```
fun calcularBalancos(): IntArray {
    val movimentosPorData = IntArray(1000)
    val balancos = IntArray(1000)
    for (movimento in lista) {
        movimentosPorData[movimento.data] +=
            if (movimento.tipo == 'r') movimento.montante else -movimento.montante
    }
    var acumulado = 0
    for (data in 0..999) {
        acumulado += movimentosPorData[data]
        balancos[data] = acumulado
    }
    return balancos
}
```

```
fun comportavel(data: Int, despesa: Int) {
    val balancos = calcularBalancos()
    for (dataCandidata in data..999) {
        var possivel = true
        for (dataFutura in dataCandidata..999) {
            if (balancos[dataFutura] - despesa < valueMinimo) { possivel = false; break }
        }
        if (possivel) { println(dataCandidata); return }
    }
    println("Incomportavel")
}
```

2.6. Testes e resultados

A sequência de comandos do exemplo do enunciado foi executada no IntelliJ (Figura 4, painel *Run*) e a saída coincide com a de referência. Por exemplo: com `m 50` o orçamento é inválido, pois o balanço na data 40 é 30; com `m 30` passa a válido; a inserção de `Mochila` (20, data 35) volta a invalidá-lo (balanço 10 na data 40); após cada `r 35 …` a listagem mostra os índices actualizados; e `w 35 20`, `w 35 3200` e `w 35 10000` devolvem, respectivamente, `60`, `990` e `Incomportavel`.

Saída obtida para o exemplo:

```
150  30  130  Invalido
150  30  130  Valido
150  10  110  Invalido
100  Invalido
0 20 d Mochila | 1 5 d Estojo | 2 5 d Caneta
0 5 d Estojo | 1 5 d Caneta | 0 5 d Estojo
130  Valido
150  60  3230  990  Incomportavel
```

3. Desafios

O trabalho consiste numa aplicação de consola interactiva, escrita em Kotlin sobre a JVM, que implementa o Sistema de Orçamento Pessoal: regista receitas e despesas (únicas ou periódicas), calcula balanços por data, verifica a validade do orçamento face a um balanço mínimo e determina a primeira data em que uma despesa é comportável.

3.1. Detalhes de implementação

Arquitectura. O código está num único ficheiro (`Main.kt`, pacote `org.emanueldossantos`) e segue a estrutura procedimental pedida no guia (Figura 5): `main()` lê as linhas da consola; `execCommand()` interpreta-as e despacha para a função do comando; as funções operam sobre o estado partilhado (`lista` e `valueMinimo`) e escrevem na consola os resultados dos comandos `l`, `b`, `v` e `w`.

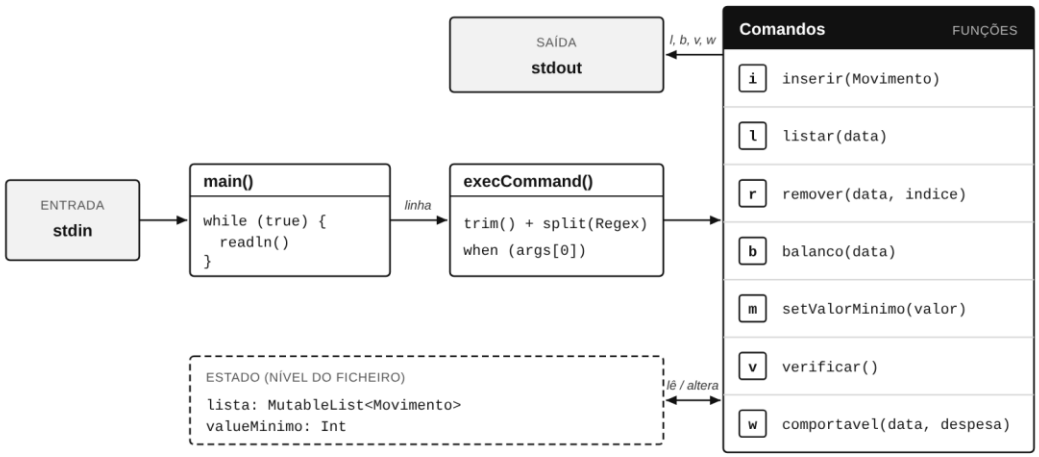


Figura 5: Fluxo de execução e organização das funções da aplicação.

Estruturas de dados. `MutableList<Movimento>` para o conjunto de movimentos (ordem de inserção preservada) e `IntArray(1000)` para os balanços diários calculados em `calcularBalancos()`. As datas são inteiros entre 0 e 999 e a periodicidade 0 indica movimento único.

Recursos de Kotlin utilizados. `val`/`var` e inferência de tipos; *data class*; expressão `when` como despacho; parâmetros nomeados; lambdas e operações de colecção (`filter`, `sumOf`, `all`, `forEachIndexed`); *string templates* (`"$indice ${movimento.montante} …"`); intervalos (`data..999`, `in movimentos.indices`); e `if` como expressão. Como o enunciado garante entradas válidas, a única verificação de erros é o ramo `else` para comandos desconhecidos; não houve necessidade de tipos anuláveis.

Limitações e evolução. O estado é global e o programa serve um único orçamento. A segunda fase do guia (refactorização orientada a objectos e vários utilizadores) pressupõe separar o código em pacotes e classes — por exemplo `Movimento`, `Orcamento`, `Utilizador` (com o `email` como chave única e a `password`) e `Sessao` (com `token` único e `datahora`) — e acrescentar o comando `u email password`, ficando `i, l, r, b, m, v, w` disponíveis apenas após o início de sessão e `exit` a encerrar a sessão em vez do programa.

4. Referências Bibliográficas

[1] Laboratório #1: Introdução a Linguagem Kotlin, João Costa, (2026, Outubro), guia da disciplina de Aplicações Móveis.

[2] Kotlin, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/home.html

[3] Hello world, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-hello-world.html

[4] Tipos básicos, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-basic-types.html

[5] Colecções, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-collections.html

[6] Controlo de fluxo, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-control-flow.html

[7] Funções, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-functions.html

[8] Classes, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-classes.html

[9] Null safety, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/kotlin-tour-null-safety.html

[10] Coding conventions, (2026, 20 de Outubro), link: https://kotlinlang.org/docs/coding-conventions.html

5. Repositório GitHub

Repositório: github.com/SW-Wanted/APLM-Laboratorios.[1]

[1] Foi enviado um convite de colaborador para o utilizador **joaojdacosta**.